

TASK: SIWE Ownership Verification for Coinbase Onramp

****Repos:**** `schnelpay-backend` (Node/TS/Express, Railway, deploys from `main`) + `schnelpay-dashboard` (Next.js, Cloudflare Pages, deploys from `main`).
****Status:**** Spec written 2026-06-14. NOT started. Scheduled for next session.
****Prereq met:**** Wallet-connect is SHIPPED and live – Convert page sources the destination from `useAccount()` (no text field). Backend already mints a proper Coinbase session token (secure-init).

WHY THIS EXISTS (the gap it closes)

Coinbase Full Access rejection asked for the destination address to be "programmatically pulled from the active cryptographic state of the user's connected wallet," and their Security Requirements doc explicitly calls for ****wallet signature verification**** (user signs a unique message proving control).

Current state after the wallet-connect ship: the address **originates** from a connected wallet in the UI, but the backend still ****trusts whatever `walletAddress` the frontend posts**** – it does not independently prove ownership. A strict reviewer (or anyone inspecting the network call) could note the endpoint would accept an arbitrary address.

SIWE closes that completely: the user signs a server-issued nonce, the ****backend verifies the signature****, and the token is minted using the address ****recovered from the signature**** – never a separately-posted field. This converts "address claimed through the UI" into "address cryptographically proven server-side," which is unambiguously what Coinbase's language reaches for. This is the bulletproof path to avoid a second rejection.

THE FLOW (EIP-4361)

1. ****Nonce (backend -> frontend):**** authenticated `GET /convert/siwe-nonce` returns a server-generated, single-use nonce stored in Redis, bound to the user, short TTL.
2. ****Sign (frontend):**** build an EIP-4361 SIWE message containing that nonce + the connected address + domain + chainId; prompt the wallet to sign it.
3. ****Verify + mint (backend):**** `POST /convert/coinbase-url` with `{ message, signature, ... }`. Backend validates the nonce (single-use), domain, chainId, expiry, and the ****signature****, extracts the address FROM the signed message, then mints the Coinbase session token with that verified address.

/!\ THE ONE CRITICAL GOTCHA – SMART WALLETS (read before writing any verify code)

The live connect modal defaults to **Coinbase Smart Wallet / "Sign in with Base"** – a **smart-contract (passkey) wallet, not an EOA**. Smart wallets do NOT sign with a plain ECDSA key, so naive ``ecrecover`` / EOA-only ``verifyMessage`` **WILL FAIL** for them. They verify via **ERC-1271** (``isValidSignature`` on the wallet contract), and if the wallet isn't deployed yet (counterfactual – common for passkey wallets before first tx), via **ERC-6492**.

Implication: server-side verification MUST go through a path that supports EOA + ERC-1271 + ERC-6492. viem's ``verifyMessage`` / ``verifySiweMessage`` does all three **when given a public client with an RPC for the wallet's chain** (it makes an ``eth_call``). So the backend needs a working RPC client per supported chain – this is a hard dependency, not optional.

Non-negotiable test: verify with BOTH a Coinbase Smart Wallet (ERC-1271/6492 path) AND an EOA like MetaMask (``ecrecover`` path). They exercise different code; both must pass. If you only test MetaMask, smart-wallet users silently break.

BACKEND CHANGES (``schnelpay-backend``)

Dependencies / config

- ``npm i viem`` (server-side; used for ``parseSiweMessage``, ``generateSiweNonce``, and the public-client ``verifySiweMessage``). Confirm exact import names against the installed viem version – viem's SIWE API lives under ``viem/siwe``.
- Public clients per supported chain with RPC transports. At minimum **mainnet (1)** and **Base (8453)** – Base because that's where Coinbase Smart Wallet lives. New env: ``RPC_URL_MAINNET``, ``RPC_URL_BASE`` (you already use Infura for Ethereum; reuse it; add a Base RPC). Set these in Railway AND locally.
- New config values: ``siweDomain`` (e.g. ``dashboard.schnelpay.com``), ``siweUri`` (e.g. ``https://dashboard.schnelpay.com``). Use env so local/staging/prod differ correctly.

Endpoint 1 – nonce (new)

- ``GET /api/v1/convert/siwe-nonce`` (auth required, reuses existing ``authenticateToken``):
- Generate a nonce (``generateSiweNonce()`` from viem, or crypto-random alphanumeric).
 - ``SETEX siwe:nonce:{userId} 600 {nonce}`` in Redis (10-min TTL).
 - Return ``{ success: true, data: { nonce } }``.

Endpoint 2 – verify + mint (change existing GET -> POST)

- ``POST /api/v1/convert/coinbase-url`` (auth required). Body: ``{ message, signature, asset?, network?, fiatCurrency?, fiatAmount? }``. Steps, in order, **fail-closed at every gate** (no token on any failure):

1. ``parseSiweMessage(message)`` -> extract ``address``, ``nonce``, ``domain``, ``chainId``, ``expirationTime``, ``issuedAt``.
2. ****Domain check:**** ``domain`` === `config.siweDomain``. Mismatch -> reject (anti-phishing/cross-site).
3. ****Nonce check (single-use):**** ``GETDEL siwe:nonce:{userId}``; must exist AND equal ``message.nonce``. Absent/mismatch -> reject. (GETDEL enforces single-use + replay protection; binding to ``userId`` ties it to the authed session.)
4. ****Expiry check:**** reject if ``expirationTime`` passed or ``issuedAt`` is implausibly old.
5. ****Signature verify:**** ``await publicClientFor(chainId).verifySiweMessage({ message, signature })`` must be ``true``. This is the EOA + ERC-1271 + ERC-6492 path. Reject on ``false`` or throw.
6. ****Use the verified address:**** the address for the token is ``parseSiweMessage(message).address`` - NEVER a separate body field. (This is the whole point.)
7. Mint via the EXISTING ``createCoinbaseSessionToken({ address: verifiedAddress, network, asset, clientId })``. ``asset``/``network`` still come from the body (validated against the existing ``ASSET_NETWORK`` map); only the ****address**** must come from the signature.
8. Return ``{ success: true, data: { url } }``.

Routing

- Update ``convert.routes.ts``: change ``coinbase-url`` from GET to POST; add the ``siwe-nonce`` GET route.
- The disabled ``transak-url`` / ``moonpay-url`` branches: leave as-is for now (those providers are off). Note in code that if reactivated they'd need the same SIWE treatment.

Interceptor / logout safety

- A failed SIWE verify may return 401/403. Confirm the dashboard's API client (``lib/api``) does NOT log the user out on these for the convert endpoints - add them to the dashboard's no-logout path list (mirror the admin ``NO_LOGOUT_PATHS`` pattern) so a rejected/cancelled signature shows an inline error instead of bouncing the user to login.

FRONTEND CHANGES (``schnelpay-dashboard`` - ``convert/page.tsx`` / ``ProviderPicker`` `handleConvert`)

- New hooks/imports: `wagmi`useSignMessage`` (``signMessageAsync``), ``useChainId``, plus `viem`createSiweMessage``.
- Rewrite ``handleConvert('coinbase')`` to:
 1. ``GET /convert/siwe-nonce`` -> ``nonce``.
 2. ``createSiweMessage({ address, chainId, domain: window.location.host, uri: window.location.origin, version: '1', nonce, statement: 'Confirm this is your receiving wallet for your SchnelPay purchase.', issuedAt: new Date() })``.
 3. ``const signature = await signMessageAsync({ message })``.
 4. ``client.post('/convert/coinbase-url', { message, signature, asset, network, fiatCurrency, fiatAmount })``.

5. Open the returned URL.

- ****Signature rejection:**** if the user cancels in the wallet, `signMessageAsync` throws – catch it, show a clean inline error ("Signature cancelled – confirm in your wallet to continue"), and do NOT proceed.
- ****Popup handling:**** keep the existing "open blank window synchronously on click, set its location after the URL is ready" pattern (preserves popup-permission from the user gesture). If the user rejects the signature, close the blank window. Verify behavior in the actual browser – the extra async signing step is the risky part for popup blockers.

SECURITY REVIEW CHECKLIST (for the Google-friend review before/after build)

- [] Nonce is server-generated, single-use (GETDEL), short-TTL, bound to the authenticated user.
- [] Token address == address recovered from the signature; no trust in any separately-posted address field.
- [] Domain and chainId are bound and checked server-side.
- [] Expiry enforced.
- [] ERC-1271 AND ERC-6492 supported (smart wallets), verified against a real RPC for the correct chain.
- [] Every failure path fails closed – no session token minted on any verification failure.
- [] Endpoint is authenticated; no addresses/secrets in URLs (POST body only).

TEST PLAN (sandbox: `COINBASE\_ENVIRONMENT=sandbox`)

1. ****Smart wallet (Coinbase Base Account):**** connect -> buy -> sign passkey -> token mints -> Coinbase flow opens. (ERC-1271/6492 path.)
2. ****EOA (MetaMask):**** same, signs with key. (ecrecover path.)
3. ****Failure cases – all must reject with NO token:**** tampered message; wrong nonce; ****reused nonce (replay)****; expired message; wrong domain.
4. Rejected signature -> clean inline error, user stays logged in, no popup left hanging.

BUILD ORDER (next session)

1. Backend: `npm i viem`; add public clients + `siweDomain`/`siweUri`/RPC config (env locally + Railway).
2. Backend: `siwe-nonce` endpoint + Redis storage.
3. Backend: `coinbase-url` GET->POST + the 8-step verify-and-mint; update `convert.routes.ts`.
4. Backend: `./node\_modules/.bin/tsc --noEmit` (from inside repo, no grep).
5. Frontend: rewrite `handleConvert` for nonce -> sign -> POST; no-logout path.

6. Frontend: ``./node_modules/.bin/tsc --noEmit``; ensure ``NEXT_PUBLIC_*` already set (they are).
7. Sandbox test: smart wallet + EOA + all failure cases.
8. Commit each repo separately (specific ``git add``, descriptive messages), push, verify on the live pages.
9. ****THEN**** resubmit Coinbase Case 01558488 for Full Access re-review – and after that, keep the integration + CDP keys STABLE through review (the Transak lesson).

CONVENTIONS (apply every step)

- Edits via Python ``str.replace()`` for TSX/TS (never `sed/heredoc/nano`).
- ``./node_modules/.bin/tsc --noEmit`` from inside each repo before commit; never ``npx tsc``; no `grep/filter`.
- ``git add`` specific files (never ``-A``); commit each repo separately.
- Secrets/keys/RPC URLs never in chat or commits – env (Railway/Cloudflare) + Bitwarden. Set new env vars in BOTH local ``.env`` and the platform (Railway for backend, CF Pages for dashboard).
- ``main`` = production for both repos; `sandbox-test` before push; re-verify on the live page after deploy.
- Style: inline styles, navy ``#2C3340`` / gold ``#C9A84C``, Playfair Display / Jost.

DEFINITION OF DONE

- Backend mints the Coinbase session token ONLY after verifying a SIWE signature, using the address recovered from that signature.
- Both smart-wallet and EOA verification paths pass in sandbox; all five failure cases reject with no token.
- Shipped to production on both repos and verified live.
- Coinbase Case 01558488 resubmitted for Full Access re-review with a note that wallet-connect + server-side ownership verification are implemented.